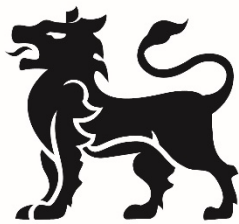


Digital Signal Processing

TUTORIAL 1

INTRODUCTION TO MATLAB PROGRAMMING FOR AUDIO



BIRMINGHAM CITY
University

1. Introduction

In this module, MATLAB is used as a tool for understanding and experimenting with **digital signals**, particularly audio signals. The aim of this tutorial is not to teach MATLAB as a general-purpose programming language, but to refresh the MATLAB skills you will need in order to work effectively with digital signal processing (DSP) concepts throughout the course.

In audio and DSP, signals are represented as sequences of numbers. MATLAB is well suited to this because it is built around vectors and matrices, which map naturally onto sampled signals. Throughout this tutorial, you should focus on the connection between:

- mathematical descriptions of signals,
- their representation as arrays in MATLAB, and
- how those arrays correspond to sound that you can listen to and visualise.

By the end of this tutorial, you should feel comfortable using MATLAB to create simple signals, inspect them visually, and understand how changes to parameters such as frequency, amplitude, and sample rate affect both the signal and the resulting sound.

2. Learning outcomes

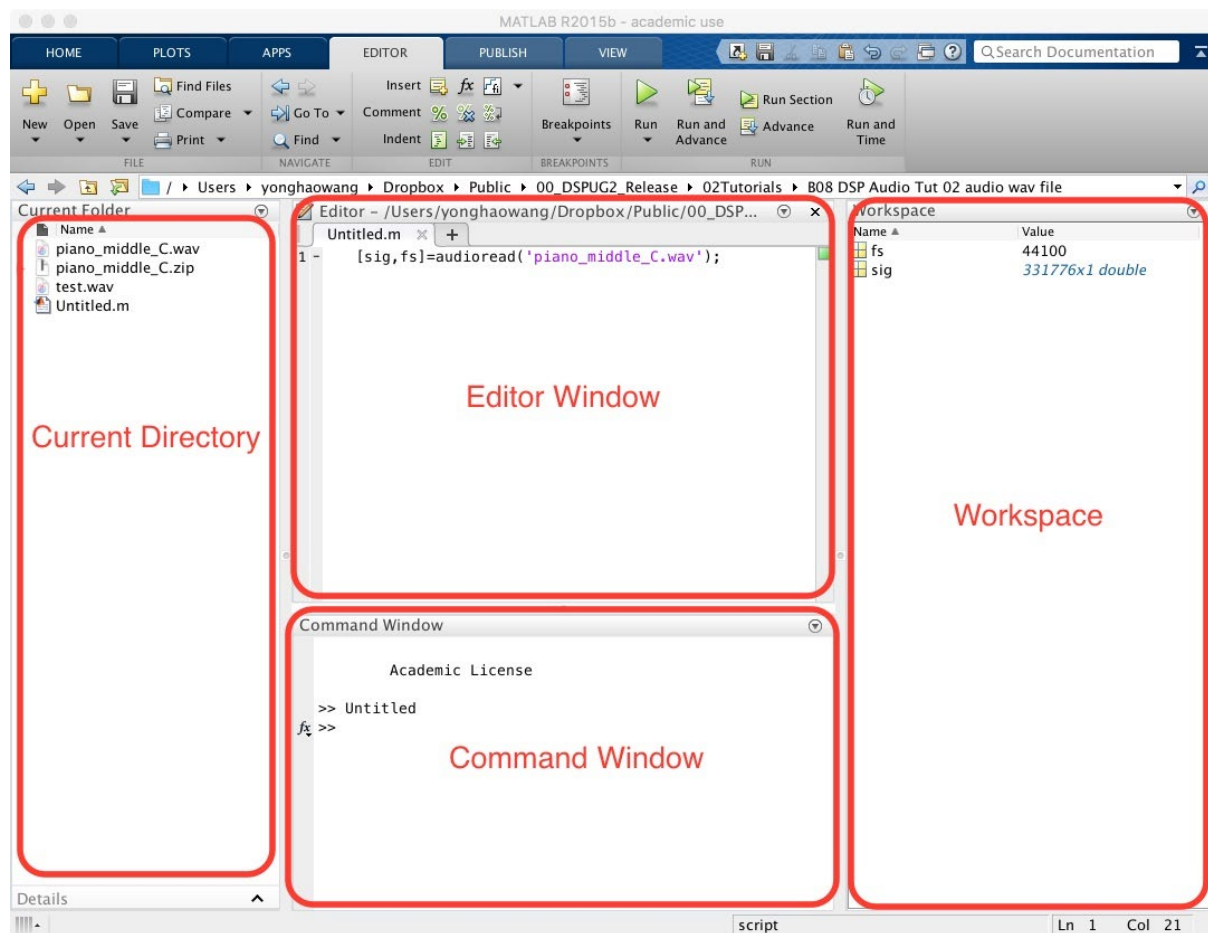
After completing this tutorial, you should be able to:

- Navigate the MATLAB environment and understand the purpose of the main windows used in DSP work.
- Write and run MATLAB scripts to carry out repeatable signal-processing tasks.
- Represent mono and stereo audio signals as arrays of samples.
- Construct a time axis from a given sample rate.
- Generate simple periodic signals such as sine waves.
- Plot signals correctly and interpret what the plots show.
- Play and save audio signals while avoiding common problems such as clipping.

These skills will be assumed knowledge in later DSP labs.

3. The MATLAB environment

When MATLAB starts, the desktop is divided into several windows. While the exact layout may differ slightly between versions, the same core components are always present. For DSP work, the most important ones are the **Command Window**, the **Editor**, the **Current Folder**, and the **Workspace**.



The **Command Window** is where MATLAB executes commands immediately. It is useful for quick calculations, checking variable values, or testing a single line of code. However, DSP work almost always involves multiple steps, so relying only on the Command Window quickly becomes impractical.

The **Editor** is where you write scripts. A script is a text file containing MATLAB commands that are executed in sequence. In DSP, scripts allow you to define an experiment: generate a signal, process it, plot it, and listen to the result. Scripts make your work repeatable, which is essential when you start comparing signals or testing the effect of different parameters.

The **Current Folder** shows the directory MATLAB is working in. Any scripts you create and any audio files you save will appear here. It is important to be aware of this location so that you know where your output files are being written.

The **Workspace** shows the variables currently stored in memory. In DSP terms, this often means vectors or matrices representing signals, as well as parameters such as sample rate or frequency. Watching the Workspace update as you run code helps reinforce the idea that signals are simply arrays of numbers.

MATLAB interface is extremely flexible and configurable. You can return it back to default layout by (Desktop -> Desktop Layout -> Default) as shown in the below diagram:

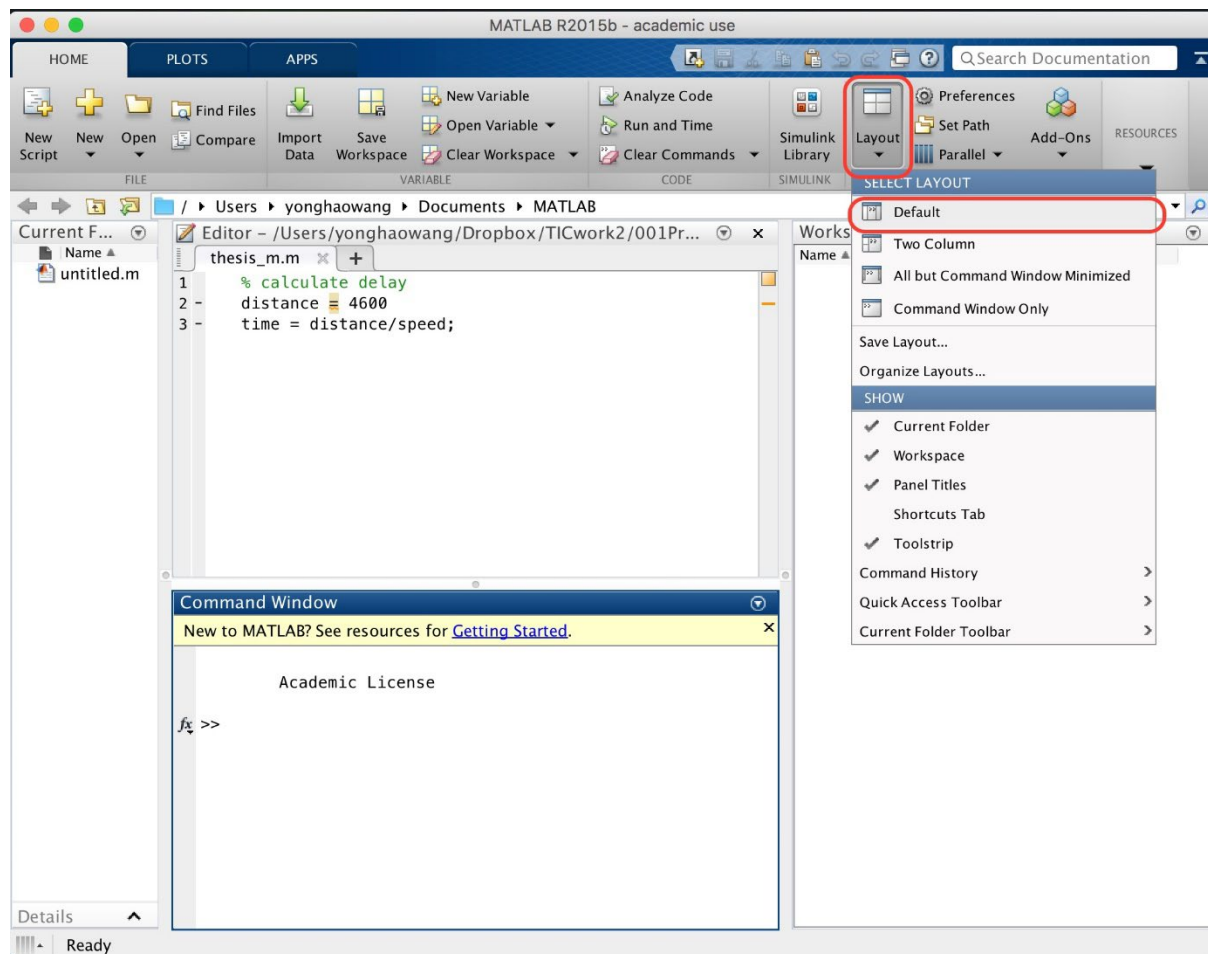


Figure 1 reset MATLAB interface to Default layout

4. Scripts and repeatable DSP experiments

In this module, you should think of a MATLAB script as a **DSP experiment**. Each time you run the script, MATLAB follows the same steps and produces the same output unless you change a parameter.

To create a script:

1. Open the Editor and create a new file.
2. Save it with a meaningful name (for example, `gensin.m` for a sine-wave generator).
3. Run the script either by pressing the Run button or by typing its name in the Command Window.

Scripts are preferred over typing commands interactively because they:

- reduce mistakes,
- allow you to revisit and modify earlier work,
- make it clear how a result was produced.

As the module progresses, scripts will become longer and more structured, but the basic idea remains the same.

5. Variables, arrays, and signals

MATLAB stores data in variables. In DSP, variables usually represent either:

- **signals** (arrays of samples), or
- **parameters** that describe those signals (such as frequency or sample rate).

Variable names must begin with a letter and can include letters, numbers, and underscores. MATLAB is case-sensitive, so `Signal` and `signal` are different variables.

Signals in MATLAB are stored as arrays:

- A **mono** audio signal is typically stored as a single vector of length N , where each element represents one sample.
- A **stereo** audio signal is typically stored as an $N \times 2$ matrix, where each column represents one channel.

This representation is not arbitrary. It matches how audio files are stored and how later DSP operations (such as filtering or mixing) are applied.

For example, a simple vector can be created as:

```
x = [1 2 3 4];
```

Operations on signals are usually **element-wise**, meaning that the operation is applied to each sample individually. MATLAB distinguishes between element-wise operations and matrix operations. For DSP work, element-wise operations are extremely common and use a dot (`.`):

```
y = x .* x;    % square each sample
```

Understanding this distinction early will prevent many errors later in the module.

6. Signals, sampling, and time

Digital signals are created by sampling a continuous-time signal at regular intervals. The number of samples per second is the **sample rate**, usually denoted f_s and measured in Hertz (Hz).

In MATLAB, it is often helpful to construct an explicit time vector so that you can relate each sample to a point in time. If n is the sample index, then the time corresponding to each sample is:

$$ts = \frac{n}{f_s}$$

This relationship appears repeatedly in DSP and is fundamental to understanding how signals are represented digitally.

7. Generating a sine wave as a signal model

One of the most important test signals in DSP is the sine wave. Sine waves are used to model pure tones and to test how systems respond at specific frequencies. In this tutorial, generating a sine wave is not just a programming exercise; it is a way to connect mathematical descriptions of signals with their sampled, digital representations.

A continuous-time sine wave can be written as:

$$x(t) = A \sin(2\pi f t)$$

where:

- A is the amplitude,
- f is the frequency in Hertz,
- t is time in seconds.

In a digital system, we do not work directly with continuous time. Instead, we evaluate the sine function at discrete time instants determined by the sample rate f_s . This is why we replace t with n/f_s , where n is the sample index.

In MATLAB, a sine-wave generator script typically follows these steps:

1. Define the sample rate.
2. Define the frequency, amplitude, and duration.
3. Create a vector of sample indices.
4. Convert sample indices to time.
5. Evaluate the sine function.

Copy the following code into the editor window and save it into your working directory with a proper filename such as “gensin.m”. Note the file name cannot have space.

```
clear; clc; close all;
Amp = 0.8;           % Amplitude (keep within [-1,1] to avoid clipping)
f    = 440;           % Frequency (Hz)
fs   = 44100;         % Sample rate (Hz)
dur  = 0.1;           % Duration (seconds)
n    = 0:round(dur*fs)-1;
t    = n/fs;
theta = 0;            % initial phase angle in radians
sig  = Amp*sin(2*pi*f*t + theta);
soundsc(sig, fs);     % Plays with automatic scaling
audiowrite('sine440.wav', sig, fs);
```

Each line in this script corresponds directly to a DSP concept. Changing any of the parameters changes the signal in a predictable way. For example:

- Increasing f increases the pitch.
- Increasing Amp increases the loudness (up to the point of clipping).
- Increasing the duration increases the number of samples.

Understanding this mapping between parameters and signal behaviour is essential for later work in filtering and frequency analysis.

8. Listening to digital signals

One advantage of working with audio signals is that you can immediately listen to the result of your code. MATLAB provides functions for audio playback that take a vector of samples and a sample rate.

To listen to the sine wave generated above, you can use:

```
soundsc(sig, fs);
```

The function `soundsc` automatically scales the signal so that it fits within the allowable range for audio playback. This helps prevent distortion when you are experimenting with amplitudes.

It is important to remember that most audio systems expect signals to lie in the range $[-1, 1]$. If your signal exceeds this range and you use `sound` instead of `soundsc`, the signal may clip, producing audible distortion. Clipping is a non-linear effect and will be discussed later in the module.

Listening to signals at this stage helps reinforce the connection between numerical arrays and perceptual sound.

Note: audio signals are typically expected to lie in the range $[-1, 1]$. If your signal clips (sounds distorted), reduce Amp . The WAV file is saved in your Current Folder (check with `pwd` and `dir`).

Sine Wave generated by MATLAB

9. Visualising signals in the time domain

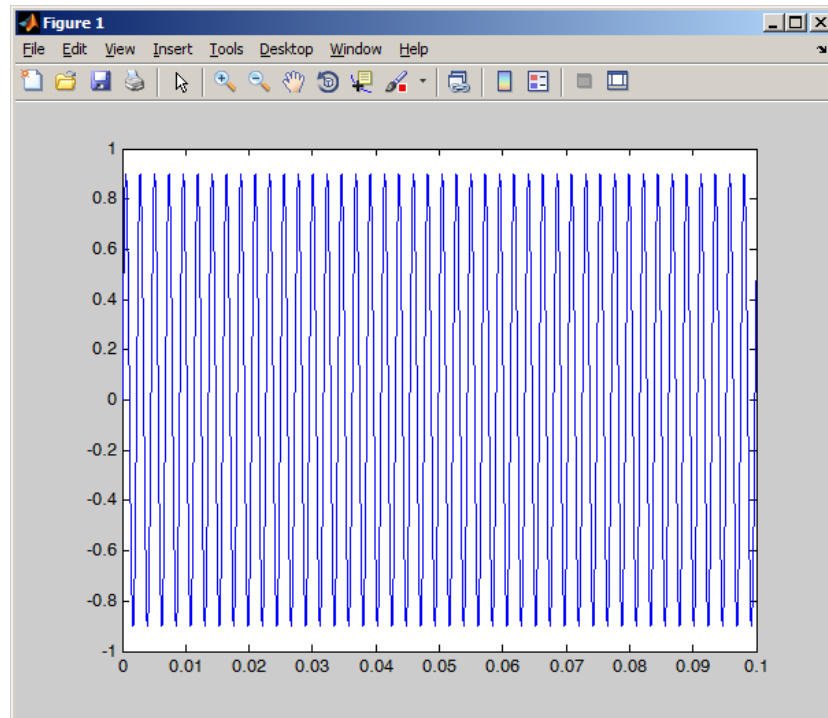
Plotting signals is a key part of DSP analysis. A plot allows you to see patterns, periodicity, and potential problems that are not obvious from listening alone.

To plot the sine wave signal against time, you can use:

```

plot(t, sig)
xlabel('Time (s)')
ylabel('Amplitude')
title('440 Hz Sine Wave')
grid on

```



The above plot shows how the signal varies over time. Even though the signal is discrete, MATLAB connects the samples with lines to make the waveform easier to interpret. If you don't include `xlabel`, `ylabel` and `title`, the plot lacks sufficient information to be of value, always label your plots.

For very short signals or when you want to emphasise the discrete nature of the data, you may also encounter the `stem` function later in the module. For now, `plot` is sufficient for visualising audio-rate signals.

When interpreting plots, always check:

- the axis labels,
- the time scale,
- whether the waveform shape matches what you expect for the given frequency and amplitude.

To create XY graphs, it is easiest to form your data into two row vectors, one for the x co-ordinates, and one for the y co-ordinates. The command

```
plot(x,y)
```


If signals are discrete typically for digital signals, the command to use is

```
stem(x, y)
```

will then create a figure with points at each y value for each matching x value. You can control the style of any line drawn through the points by a third string argument to the plot command:

```
plot(x, y, style);
```

where style is made up from characters as follows:

- Color strings are 'c', 'm', 'y', 'r', 'g', 'b', 'w', and 'k'. These correspond to cyan, magenta, yellow, red, green, blue, white, and black.
- Linestyle strings are '-' for solid, '--' for dashed, ':' for dotted, '-.' for dash-dot, and none for no line.
- The marker types are '+', 'o', '*', and 'x' and the filled marker types 's' for square, 'd' for diamond, '^' for up triangle, 'v' for down triangle, '>' for right triangle, '<' for left triangle, 'p' for pentagram, 'h' for hexagram, and none for no marker.

10. Saving audio signals to a file

In addition to listening to signals inside MATLAB, you will often want to save them as audio files so that they can be used elsewhere or submitted as part of coursework.

MATLAB can write WAV files using the `audiowrite` function:

```
audiowrite('sine440.wav', sig, fs);
```

This command saves the signal to a file named `sine440.wav` in the **Current Folder**. It is good practice to use descriptive filenames that include relevant information such as frequency or processing method.

If you cannot find the saved file, check:

- the Current Folder panel in MATLAB,
- the output of the `pwd` command to see the current directory.

Saving audio reinforces the idea that MATLAB signals and audio files are two representations of the same underlying data.

11. Common issues and good practice

At this stage, several common issues tend to arise:

- **No sound or very quiet sound:** check the amplitude of your signal.
- **Distorted sound:** check for clipping and ensure amplitudes remain within $[-1, 1]$.
- **Unexpected plots:** verify the time vector and sample rate.
- **Missing files:** confirm the Current Folder before saving.

Developing the habit of checking parameters and inspecting plots early will save time as the DSP tasks become more complex.

Useful Built-in functions

Generation

<code>zeros()</code>	matrix of specified size filled with zeros
<code>ones()</code>	matrix of specified size filled with ones
<code>rand()</code>	generate pseudo random number(s) between 0 and 1

Arithmetic

<code>rem()</code>	remainder after integer division
<code>abs()</code>	absolute value (also character -> number)
<code>fix()</code>	truncate a value to its integer part (towards zero)
<code>round()</code>	round a value to nearest integer.
<code>sqrt()</code>	square root
<code>sin()</code>	sine (angle in radians)
<code>cos()</code>	cosine (angle in radians)
<code>exp()</code>	exponential
<code>log()</code>	natural logarithm
<code>log10()</code>	logarithm base 10

Status

<code>length()</code>	length of a vector (longest dimension of matrix)
<code>size()</code>	size of a matrix [nrows, ncols]

Miscellaneous

<code>sum()</code>	sum the elements of a vector
<code>mean()</code>	find mean of elements of a vector
<code>sort()</code>	sort the elements of a vector in increasing size
<code>clock()</code>	returns date and time as a vector [year month day hour minute seconds]
<code>date()</code>	returns date as a string dd-mmm-yyyy

12. Exercises: MATLAB as a DSP tool

The following exercises consolidate the ideas introduced in this tutorial.

1. Write a MATLAB script that generates a 1 kHz sine wave with a duration of 0.5 seconds at a sample rate of 44.1 kHz.
2. Plot the signal and label the axes correctly.

3. Play the signal and adjust the amplitude so that it is clearly audible but not clipped.
4. Save the signal as a WAV file with an appropriate filename.
5. Modify the script to generate a stereo signal where:
 - the left channel is a 500 Hz sine wave,
 - the right channel is a 1 kHz sine wave.
6. Plot the first 500 samples of each channel on separate subplots.
7. Modify the amplitude of the signal so it decreases linearly with time. E.g. Amplitude 1 to 0 over the duration of the audio.
8. Reverse the audio and save the output.
9. Switch left and right channels, confirm with headphones
10. Covert audio to mono by averaging left and right channels.

These exercises prepare you for later labs where you will analyse, modify, and compare signals using DSP techniques such as filtering and frequency-domain analysis.

1. Additional arithmetic Exercises

For these exercises, use the editor window to enter your code, and save your answers to files under your account on the central server. To run the programs, use the Debug/Run menu option or cut and paste the code into the command window. When you save the files, give them the file extension of ".m". Solution of Question 1) is given below. Solutions will be available at end of tutorial session.

- 1) Write a program to assign the following expressions to a variable A and then to print out the value of A.
 - a) $(2*3 + 1) / 5$
 - b) $(3.2e-5 + 7.1e3) / (4.6e-2)$
 - c) $\sin(2*\pi*50*0.01) + \cos(\pi/3)^2$

For each part, assign the expression to A and then display A using disp(A).

d) $(0.0000123 + 5.67 \times 10^{-3}) \times 0.4567 \times 10^{-4}$

- 2) Celsius temperatures can be converted to Fahrenheit by multiplying by 9, dividing by 5, and adding 32. Assign a variable called C the value 37, and implement this formula to assign a variable F the Fahrenheit equivalent of 37 Celsius.
-